

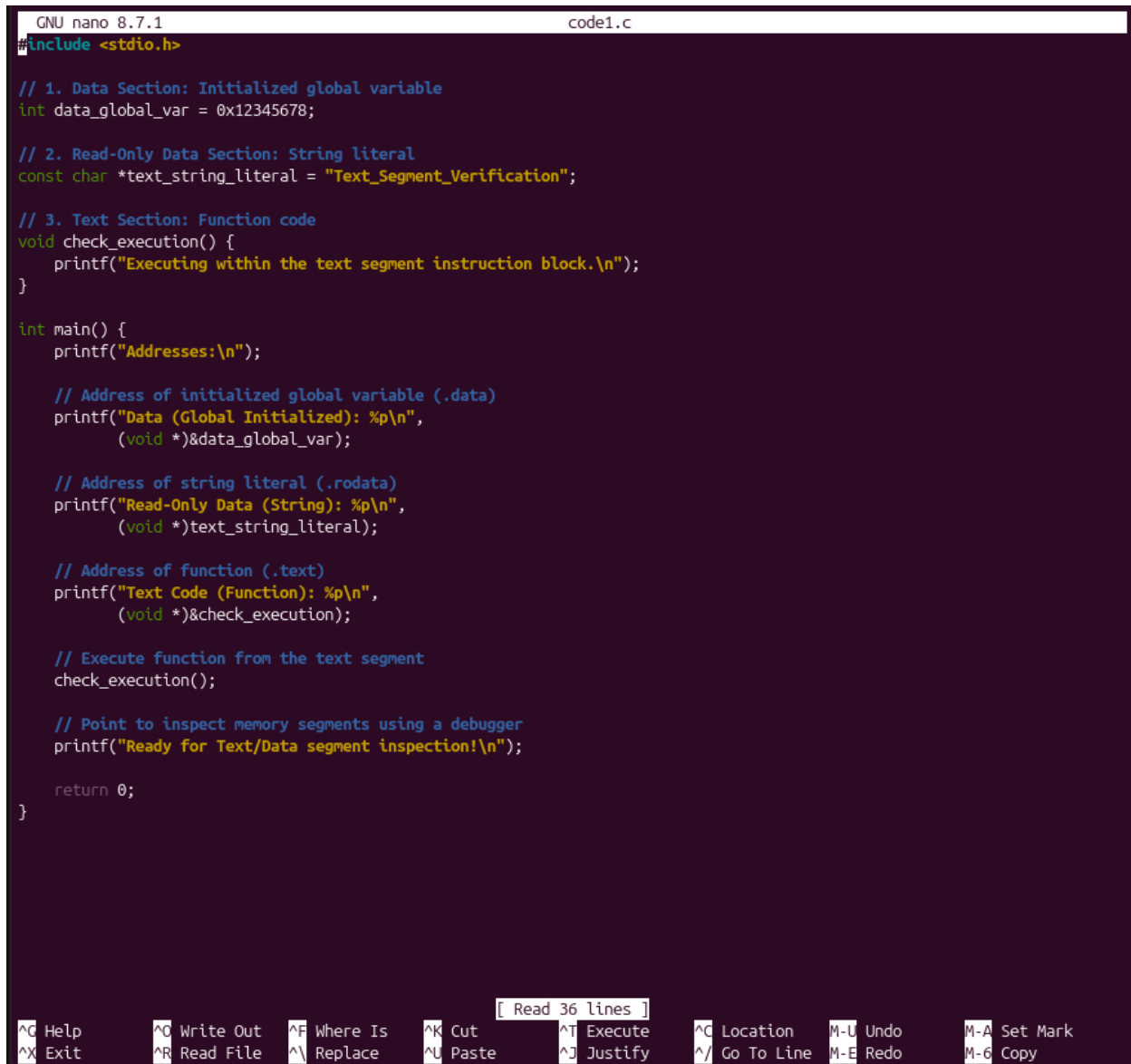
Table of Contents:

Chapter 3: TARGET LABORATORY SOURCE CODE	2
Chapter 4: LABORATORY PROCEDURES AND TASKS	3

Table of Figures:

Figure:Source Code	3
Figure: Building Debugging Symbols.	4
Figure: Setting breakpoint and runtime positioning.	5
Figure: Memory layouts observation.	6
Figure: Clean termination of the runtime environment.	6

Chapter 3: TARGET LABORATORY SOURCE CODE



```
GNU nano 8.7.1                                code1.c
#include <stdio.h>

// 1. Data Section: Initialized global variable
int data_global_var = 0x12345678;

// 2. Read-Only Data Section: String literal
const char *text_string_literal = "Text_Segment_Verification";

// 3. Text Section: Function code
void check_execution() {
    printf("Executing within the text segment instruction block.\n");
}

int main() {
    printf("Addresses:\n");

    // Address of initialized global variable (.data)
    printf("Data (Global Initialized): %p\n",
        (void *)&data_global_var);

    // Address of string literal (.rodata)
    printf("Read-Only Data (String): %p\n",
        (void *)&text_string_literal);

    // Address of function (.text)
    printf("Text Code (Function): %p\n",
        (void *)&check_execution);

    // Execute function from the text segment
    check_execution();

    // Point to inspect memory segments using a debugger
    printf("Ready for Text/Data segment inspection!\n");

    return 0;
}
```

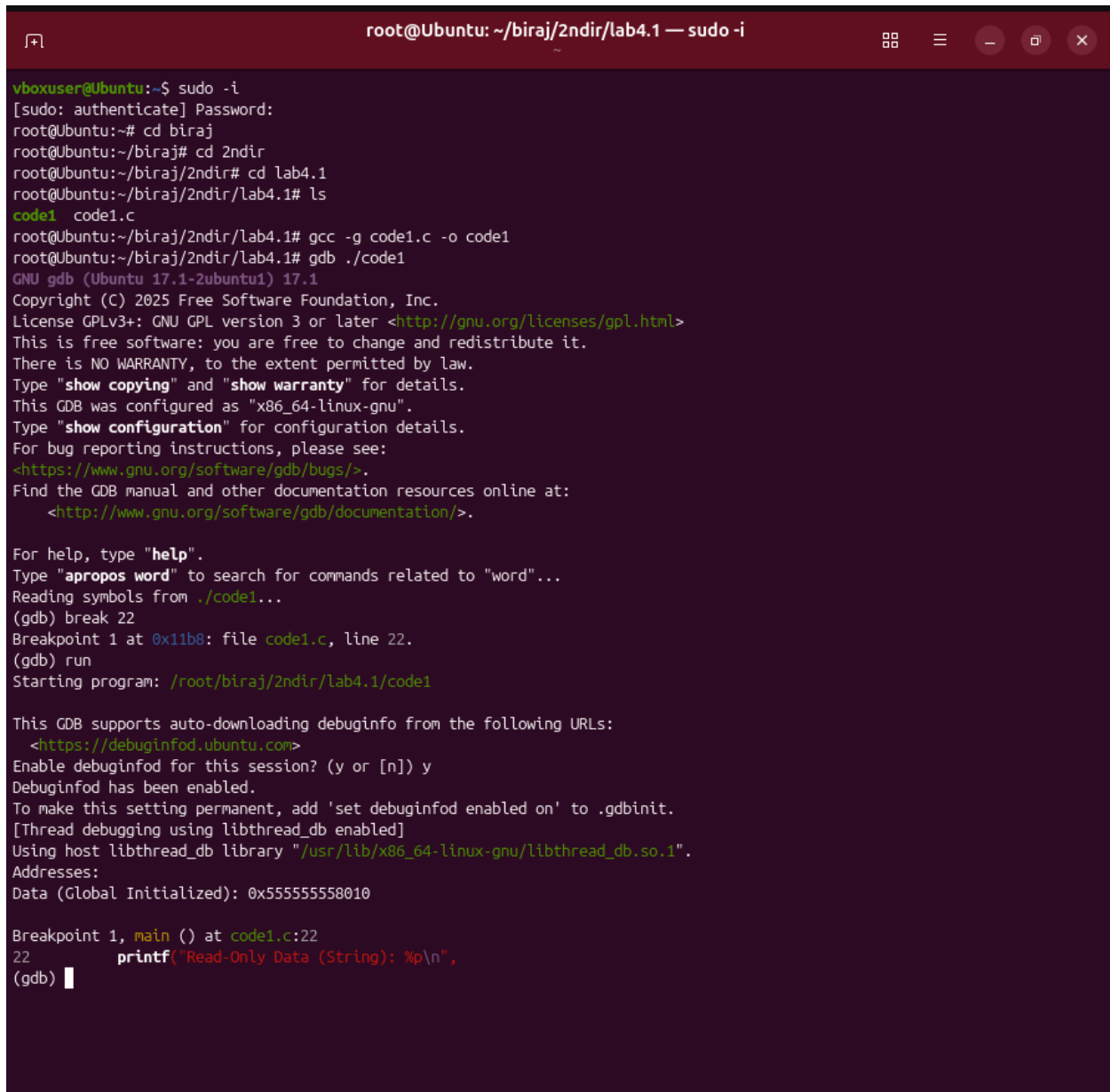
[Read 36 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

Figure:Source Code

Summary: The file type elf 64-littleaarch64 indicates a 64-bit AArch64 system using little-endian byte ordering. The check_execution() address falls within the .text range, the string address falls within .rodata, and the initialized global variable falls within .data. The .data section is located at a lower address than .bss.

Chapter 4: LABORATORY PROCEDURES AND TASKS



```
root@Ubuntu: ~/biraj/2ndir/lab4.1 — sudo -i

vboxuser@Ubuntu:~$ sudo -i
[sudo: authenticate] Password:
root@Ubuntu:~# cd biraj
root@Ubuntu:~/biraj# cd 2ndir
root@Ubuntu:~/biraj/2ndir# cd lab4.1
root@Ubuntu:~/biraj/2ndir/lab4.1# ls
code1  code1.c
root@Ubuntu:~/biraj/2ndir/lab4.1# gcc -g code1.c -o code1
root@Ubuntu:~/biraj/2ndir/lab4.1# gdb ./code1
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./code1...
(gdb) break 22
Breakpoint 1 at 0x11b8: file code1.c, line 22.
(gdb) run
Starting program: /root/biraj/2ndir/lab4.1/code1

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized): 0x555555558010

Breakpoint 1, main () at code1.c:22
22      printf("Read-Only Data (String): %p\n",
(gdb) |
```

Figure: Building Debugging Symbols.

Summary: A C file named “code1” was created and the source code was written which was later compiled and using the gdb debugger the files was peer inside.

```
root@Ubuntu: ~/biraj/2ndir/lab4.1 — sudo -i
Reading symbols from ./code1...
(gdb) break 22
Breakpoint 1 at 0x11b8: file code1.c, line 22.
(gdb) run
Starting program: /root/biraj/2ndir/lab4.1/code1

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized): 0x555555558010

Breakpoint 1, main () at code1.c:22
22      printf("Read-Only Data (String): %p\n",
(gdb) info files
Symbols from "/root/biraj/2ndir/lab4.1/code1".
Native process:
Using the running image of child process 4617.
While running this, GDB does not access memory from...
Local exec file:
'/root/biraj/2ndir/lab4.1/code1', file type elf64-x86-64.
Entry point: 0x555555558080
0x0000555555554350 - 0x00005555555554374 is .note.gnu.build-id
0x00005555555554374 - 0x00005555555554390 is .interp
0x00005555555554390 - 0x000055555555543b4 is .gnu.hash
0x000055555555543b8 - 0x00005555555554478 is .dynsym
0x00005555555554478 - 0x0000555555555450c is .dynstr
0x0000555555555450c - 0x0000555555555451c is .gnu.version
0x00005555555554520 - 0x00005555555554550 is .gnu.version_r
0x00005555555554550 - 0x00005555555554628 is .rela.dyn
0x00005555555554628 - 0x00005555555554658 is .rela.plt
0x00005555555555000 - 0x0000555555555501b is .init
0x00005555555555020 - 0x00005555555555050 is .plt
0x00005555555555050 - 0x00005555555555060 is .plt.got
0x00005555555555060 - 0x00005555555555080 is .plt.sec
0x00005555555555080 - 0x0000555555555520f is .text
0x00005555555555210 - 0x0000555555555521d is .fini
0x00005555555556000 - 0x000055555555560e8 is .rodata
0x000055555555560e8 - 0x00005555555556124 is .eh_frame_hdr
0x00005555555556128 - 0x000055555555561f4 is .eh_frame
0x000055555555561f8 - 0x00005555555556228 is .note.gnu.property
0x00005555555556228 - 0x00005555555556248 is .note.ABI-tag
0x00005555555557db0 - 0x00005555555557db8 is .init_array
0x00005555555557db8 - 0x00005555555557dc0 is .fini_array
0x00005555555557d40 - 0x00005555555557f60 is .dynamic
```

Figure: Setting breakpoint and runtime positioning.

Summary: With the help of gdb debugger and its commands addresses containing the specific strings were confirmed.

```

Reading symbols from ./code1...
(gdb) print &data_global_var
$1 = (int *) 0x4010 <data_global_var>
(gdb) x/wx &data_global_var
0x4010 <data_global_var>:      0x12345678
(gdb) print text_string_literal
$2 = 0x2008 "Text_Segment_Verification"
(gdb) x/s text_string_literal
0x2008: "Text_Segment_Verification"
(gdb) print &text_string_literal
$3 = (const char **) 0x4018 <text_string_literal>
(gdb)

```

Figure: Memory layouts observation.

Summary: This demonstrates an important distinction: `text_string_literal` points to the string stored in `.rodata`, while the pointer variable itself is stored in `.data`.

```

(gdb) break 22
Breakpoint 1 at 0x555555551b8: file code1.c, line 22.
(gdb) run
Starting program: /root/biraj/2ndir/lab4.1/code1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized): 0x555555558010

Breakpoint 1, main () at code1.c:22
22      printf("Read-Only Data (String): %p\n",
(gdb) continue
Continuing.
Read-Only Data (String): 0x555555556008
Text Code (Function): 0x55555555169
Executing within the text segment instruction block.
Ready for Text/Data segment inspection!
[Inferior 1 (process 4820) exited normally]
(gdb) quit
root@Ubuntu:~/biraj/2ndir/lab4.1#

```

Figure: Clean termination of the runtime environment.